

NEUROSCOPE: Reverse Engineering Deep Neural Network on Edge Devices using Dynamic Analysis

Overview

The development in the field of edge devices (e.g. IoT devices) is surging continuously. The technology at the intersection of edge devices and Artificial Intelligence has seen a rapid growth recently. Deep Neural Network (DNN) models are gaining popularity as one of the main software components of edge devices. Their use has become very prevalent across different types of edge devices. In this usage scenario, the inference phase of the DNN model is executed by a dedicated, compiled piece of code (i.e., a DNN binary). However, these models are vulnerable to being reverse engineered, thus leading to stealing DNN models, gray/white-box adversarial machine learning attacks and defenses, and backdoor detection.

In this report, we propose **NEUROSCOPE**, a novel data-driven approach based on dynamic analysis and machine learning to reverse engineer DNN binaries. We have followed a compiler-independent and code-feature free approach that enables NEUROSCOPE to support a larger variety of DNN binaries across different DNN compilers and hardware platforms, including binaries implementing DNN models using an interpreter-based approach. We demonstrate NEUROSCOPE's capability by using it to reverse engineer DNN binaries unsupported by previous approaches with high accuracy.

Introduction

Deep Neural Networks (DNNs) are a category of machine learning algorithms that extract and transform features using a series of cascaded DNN operators including convolution, pooling, and activation. Each DNN operator receives the output of preceding operators as its input and calculates its output based on its operator type and attributes.

In recent years, with Deep Learning gaining boom, there has been an attempt to perform a variety of tasks such as image recognition, natural language processing, and autonomous driving using Deep Neural Network (DNN). Some of these models are deployed on high-end GPU servers (and are accessible online), and some are deployed locally on edge devices, such as IoT devices. A smart-camera performing on-device face recognition is the best

example. In such cases, the inference phase of a DNN is executed by a dedicated, compiled piece of code, called a DNN binary which can be reverse engineered to steal DNN models, which are considered valuable intellectual properties. Traditional, general-purpose de-compilers are incapable of reverse engineering the DNN models since they can only recover a C-like representation of the code.

The reverse engineering of DNN models involves identification of the DNN operators implemented in the DNN binaries and then recovery of the high level representation of implemented DNN. These DNNs can run in two ways on a device – **Ahead-of-Time** approach (each operation like convolution, ReLU, pooling becomes a separate function in the binary) and **Interpreter-based approach** (the model is stored as data weights + structure in some proprietary format).

The current approaches to reverse engineer a DNN model only target **Ahead-of-Time** approach. Hence, the existing approaches, which rely on code features (either extracted by symbolic execution or machine learning techniques), cannot extract a high-level representation of the DNNs from these binaries. Reliance on specific code features also forces current approaches to retrain the machine learning models or develop additional compiler-specific features for each DNN compiler, CPU architecture, or even compiler version.

To address these limitations, we have implemented NEUROSCOPE, a novel data-driven approach based on dynamic analysis to reverse engineer DNN binaries. NEUROSCOPE is efficient in reverse engineering the DNN binaries compiled using both Ahead-of-Time and Interpreter-based approach. We have observed that, although DNNs may be implemented differently by various compilers on different hardware platforms, the mathematical semantics of the DNN operators remain the same. Therefore, by examining the data that different operators receive and output, we can infer the semantics of operators in hardware-independent fashion.

NEUROSCOPE first identifies the DNN operators implemented in the binary using dynamic analysis, and then captures pairs of input/output data, i.e., pairs of data being processed by and returned from each DNN operator. Then, it uses dedicated machine learning models to infer the DNN semantics from input/output pairs enriched with statistical features extracted from the input/output pairs.

Challenges To Address:

Challenge 1: Supporting diverse and proprietary DNN SDKs and hardware platforms. DNNs are compiled and deployed into different hardware platforms, with different vendor-specific SDKs.

Solution 1: Even though DNN implementations vary across compilers and hardware, the underlying mathematical behaviour of their operators remains the same and can be inferred from input–output patterns, which are largely independent of SDKs and platforms.

Challenge 2: Inferring semantics from input/output pairs with unknown tensor shape, operator attributes, and parameters. We infer DNN operator semantics from input–output relationships, but this is challenging due to unknown tensor shapes, parameters, and the impracticality of exhaustively testing all possible operators.

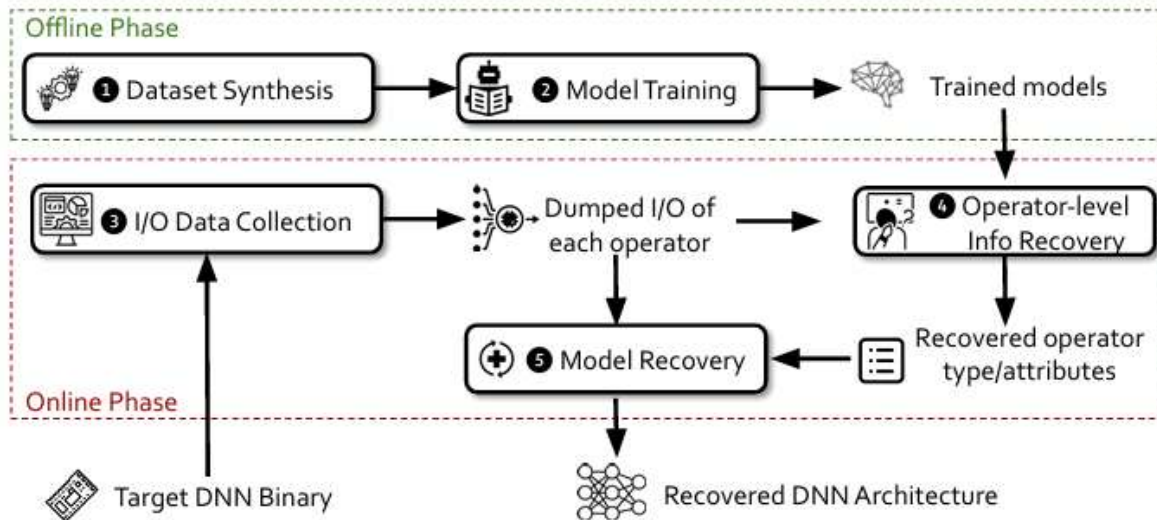
Solution 2: We use a Seq2Seq model to learn DNN semantics from input–output pairs, enhancing embeddings with statistical features to predict operator types and attributes, and train it using synthesized data for various operators.

Challenge 3: Part of the DNN may be executed using hardware accelerators. DNN computation graph is divided into multiple subgraphs, each containing multiple operators that are supported by the same backend. Some subgraphs of the DNN may be executed using hardware accelerators available on the edge device. The internal function of these accelerators is a black box, and we can only observe the input and output of the whole subgraph, instead of the intermediate states of each operator, preventing us from inferring the semantics of each individual DNN operator executed by a hardware accelerator.

Solution 3: Current DNN inference runtimes usually include the CPU backend code for all the operators in the binary even when hardware accelerators are available. In this way, when hardware accelerators are not available, the runtime can robustly fall back to the CPU backend for subgraph execution, where we can observe the input/output of each operator in the subgraph. Observing this, we use various approaches to prevent the runtime from using the hardware accelerators, forcing it to fall back to the CPU backend for subgraph execution, and can then infer the semantics of each operator in the subgraph.

NEUROSCOPE:

NEUROSCOPE supports recovering DNN architecture (i.e., operator types, attributes, and topology) from DNN binaries and outputs the recovered architecture in ONNX format.



Overview of NEUROSCOPE pipeline

NEUROSCOPE consists of two phases – an offline phase and an online phase.

In offline phase, NEUROSCOPE first generates the training dataset consisting of numerous input/output pairs. It then trains a classification neural network to recover the operator type, and a set of dedicated regression neural networks, to recover the operator attributes.

In the online phase, NEUROSCOPE analyzes the target DNN binary to reconstruct its architecture using previously trained models. It first identifies where each operator’s computations occur in the binary and locates their corresponding input and output buffers. The data from these buffers is then extracted and fed into trained models to determine the operator type and its attributes. Finally, by examining how data flows between these buffers, NEUROSCOPE rebuilds the overall DNN topology, capturing how different operators are connected.

The offline phase is conducted only once to train models, and the online phase is conducted for each run of the DNN binaries (i.e., once per input).

Offline Phase

This phase consists of two steps – dataset synthesis and model training.

1. Dataset Synthesis

To synthesize the dataset, NEUROSCOPE repeatedly chooses a DNN operator type, generates random input tensor shape, operator attributes, and operator parameters, and it computes the corresponding output tensor. It then stores the input/output tensor pairs along with their operator types, attributes, and parameters in the dataset.

2. Model Training

This section first outlines the challenges in framing DNN operator recovery as a learning problem, and then describes the models used for this task. The objective is: given an input–output tensor pair of an operator, predict both its type and its attributes.

A key challenge arises from the complexity of treating this as a single multi-class classification problem. If operator type and all attribute combinations are predicted together, the number of classes becomes extremely large. For example, a convolution operator has multiple attributes (like kernel size, stride, padding, channels), each with many possible values—leading to an exponential number of combinations, which makes training difficult and inefficient.

To address this, a divide-and-conquer strategy is used. Instead of one large model, the task is split into two stages. First, a classification model predicts the operator type. Then, based on this predicted type, separate regression models are used to estimate each attribute individually. This simplifies the learning process and improves accuracy, as each model focuses on a smaller, more manageable task.

The second challenge is modeling the relationship between input and output tensors. Unlike standard tasks that process a single sequence, this problem requires understanding the numerical dependency between two sequences. While Seq2Seq models with attention are commonly used for such tasks, attention mechanisms become computationally expensive for very long sequences (e.g., large tensor sizes). On the other hand, using Seq2Seq without attention fails to capture long-range dependencies effectively, leading to poor accuracy.

To address this, the approach enhances Seq2Seq with additional statistical features extracted from the input and output tensors. These features help capture important patterns that the model might otherwise miss. For instance, counting the number of zeros in the output tensor can help distinguish between operations like Conv and

Conv+ReLU, since ReLU introduces more zeros by converting negative values to zero. By combining learned representations with such statistical cues, the model can more accurately infer operator types and attributes.

Model Architectures

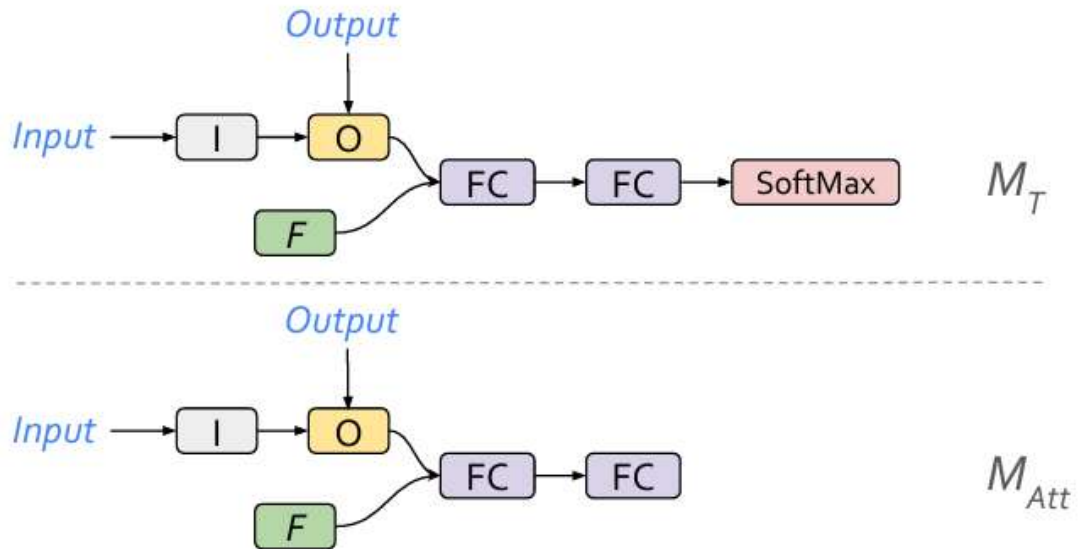


Figure above illustrates the architectures of MT (classification neural network) and MAtt (regression neural network). In both models, the input tensor is first processed by an LSTM layer, and its final hidden state is used to initialize a second LSTM layer that processes the corresponding output tensor. This sequential setup helps capture the relationship between input and output at the element level, where each value is treated as a floating-point sequence.

The final state from the second LSTM is then combined with statistical features extracted from the input/output tensors and passed through two fully connected (FC) layers. For MT, the final layer outputs class probabilities (via Softmax) to predict the operator type, while for MAtt, it outputs a single value representing a specific attribute.

This design is inspired by Seq2Seq-based program synthesis methods but improves upon them by incorporating statistical features. Since Seq2Seq alone struggles to fully capture input–output dependencies, combining it with these features and FC layers leads to better accuracy.

Online Phase

It is conducted on the target DNN binary on the edge device to recover the DNN architecture. It consists of three steps – I/O data collection, Operator-level info recovery, and Model recovery.

1. I/O Data Collection

In this section, we will explore how NEUROSCOPE collects input and output data from DNN operators. This data is later used to recover details about each operator and reconstruct the overall DNN model. The data collection process involves two main steps:

A) Forcing CPU execution:

To make the execution of DNN operators observable, NEUROSCOPE forces the runtime to run on the CPU instead of a hardware accelerator. It does this by hiding the accelerator from the machine learning runtime.

B) Locating operators and buffers:

After that, NEUROSCOPE identifies where each DNN operator is implemented and finds the memory buffers that store their input and output data.

2. Operator-Level Info Recovery

NEUROSCOPE uses the models M_t (for operator type) and M_{att} (for attributes), which were trained during the offline phase, to identify each DNN operator and its properties from the collected input/output buffer data.

First, NEUROSCOPE runs the target DNN binary with N different inputs. For each operator, it collects N sets of input/output buffer data. Then, it feeds each of these buffer dumps into the M_t model to predict the operator type. The final operator type is decided using majority voting across the N predictions. If there is a tie, one of the top-voted votes is chosen randomly.

Once the operator type is identified, NEUROSCOPE uses the M_{att} model to predict the operator's attributes. It again uses the same N buffer dumps and applies majority voting to determine the final attribute values.

Not all the attributes need to be predicted directly. Some attributes can be derived from others or from previous operators. For example, in a convolutional layer, the number of input channels is equal to the number of output channels of the previous layer. Therefore, NEUROSCOPE only

predicts a subset of important attributes, and the rest are inferred from them. To recover the selected attributes, NEUROSCOPE inputs the collected buffer data into corresponding Matt model and rounds the predicted values to obtain the final results.

3. Model Recovery

In the final step, NEUROSCOPE reconstructs the complete DNN model by combining the recovered operator details with the overall network structure.

A) Recovering DNN Topology:

NEUROSCOPE determines the structure of the DNN by analyzing how data flows between operators. It tracks the memory addresses of input and output buffers for each operator. If the input of one operator matches the output of another, NEUROSCOPE connects them. By repeating this process for all operators and considering their execution order, it reconstructs the full network topology.

B) Recovering DNN Model:

After recovering the topology, NEUROSCOPE combines it with the previously identified operator types and their attributes. It then generates a high-level representation of the entire DNN model in the ONNX format.

Implementation

1. Dataset and Models

The dataset generation and neural network models are built using PyTorch. NEUROSCOPE currently supports 12 common DNN operators, including Add, Conv, Relu, LSTM, RNN, Softmax, and others. These operators are widely used in popular models like ResNet and MobileNet. For model design, the hidden size of the LSTM is set to 16, and the first fully connected layer has size 32. These values were selected using grid research over options (8, 16, 32, 64), choosing the best-performing configuration on validation data. Mt is trained using cross-entropy loss, while Matt uses mean squared error. Both models use the Adam optimizer with a learning rate of 0.001.

2. Static Analysis

Static analysis is performed using angr to identify loop structures and key instructions (such as memory access and vector operations). This

analysis is applied to firmware binaries (for bare-metal devices) or shared libraries (for OS-based devices).

3. Dynamic Analysis

For runtime analysis, NEUROSCOPE uses ptrace on Linux-based devices to set breakpoints and monitor memory access. On bare-metal devices, it uses SEGGER J-Link with GDB support.

Evaluation

1. Synthesized Dataset and Training

100,000 input/output pairs for each operator were synthesized. In total, the synthesized dataset contains 1.2 million pairs of input/output tensors. We split the synthesized dataset into training, validation, and testing datasets with a ratio of 8:1:1. Models were trained on the training dataset and validated on the validation dataset. Models were trained for 30 epochs with batch size 128, and the model with the best overall validation accuracy was selected as the final model.

2. Evaluation on Testing Synthesized Dataset

We evaluate the performance of our trained models on the testing dataset.

